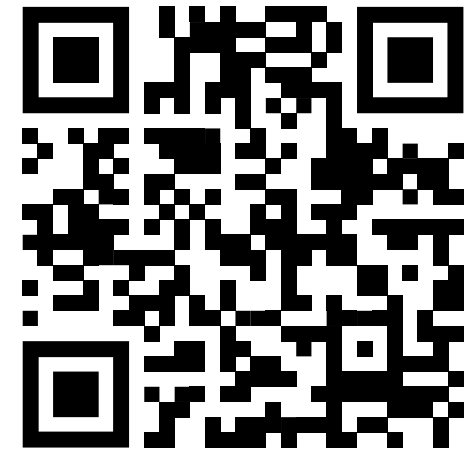


Programmierung in C++

Programmierkonzepte, Standards, Best-Practices

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Vinzenz Funk



<https://poll.hs-kempten.de/poll/>

Programmierkonzepte, Standards, Best-Practices



Einführung

- Bei der Programmierung mit C++ haben sich über die Zeit verschiedene **Praktiken** etabliert mit der Sprache auf Basis des Standards umzugehen
 - Diese dienen vor allem der Reduzierung von Programmierfehlern aber auch der besseren Übersichtlichkeit und Wartbarkeit eines C++-Programms
- Durch die Modernisierung der Sprache mit den **C++11**, **C++14**, **C++17**, **C++20** und **C++23** Standards, haben sich weitere Praktiken ergeben, die Entwicklung von C++-Anwendungen sicherer zu gestalten
 - Das Verständnis moderner Sprachkonzepte ist essenziell für die zukunftsfähige Entwicklung von C++-Anwendungen
- Dieses Kapitel befasst sich mit einzelnen Programmierkonzepten, Standards und Best-Practices, zwischen denen nicht zwingend ein Zusammenhang besteht



Aliase

Lernziele

- Anwendungszwecke und Vorteile der Nutzung von **Typ-Aliassen** wurden verstanden
- Aliase können in eigenen Projekten *auf moderne Art* eingesetzt werden
 - Dazu wird der Unterschied zwischen **typedef**- und **using**-Aliassen exemplarisch betrachtet

Aliase



Motivation

- Häufig entstehen – gerade bei der Verwendung von Template-Objekten – **lange Typnamen mit verschachtelten Deklarationen**
 - Das ist besonders **wartungsintensiv** wenn diese Deklarationen häufig wiederholt werden müssen (Header- & Quelldatei, Variablen, Rückgabewerte, ...)
- Beispiel: Mehrfache Verwendung von Smart-Pointern als Rückgabetyt von Methoden:

```
class TextureFactory
{
public:
    std::shared_ptr<Texture> getOrCreateWallTexture();
    std::shared_ptr<Texture> getOrCreateGroundTexture();
    // ...

private:
    std::shared_ptr<Texture> m_textureWall;
    std::shared_ptr<Texture> m_textureGround;
};
```

Aliase

Motivation



- Für derartige Deklarationen bieten sich **Aliase** (Alternativnamen, Stellvertreter) an
 - Dabei wird ein anderer Name stellvertretend für die (u.U. ausführliche) Typdeklaration verwendet
- Beispiel: Ersatz der Smart-Pointer-Deklarationen mit einem Alias

```
class TextureFactory
{
public:
    using TexturePtr = std::shared_ptr<Texture>;

    TexturePtr getOrCreateWallTexture();
    TexturePtr getOrCreateGroundTexture();
    // ...

private:
    TexturePtr m_textureWall;
    TexturePtr m_textureGround;
};
```

Definition des Typ-Alias „TexturePtr“
für `std::shared_ptr<Texture>`

Verwendung von `TexturePtr`
anstelle der ausführlichen
`std::shared_ptr<Texture>`
Deklaration

Aliase



Motivation

- Neben der verminderten Schreibarbeit und Fehleranfälligkeit, können Typen durch Aliase einfach ersetzt werden
→ **geringerer Wartungsaufwand**
- Beispiel: Ersatz des **shared_ptr** mit einem **weak_ptr**

```
class TextureFactory
{
public:
    std::weak_ptr<Texture> getOrCreateWallTexture();
    std::weak_ptr<Texture> getOrCreateGroundTexture();
    // ...
private:
    std::weak_ptr<Texture> m_textureWall;
    std::weak_ptr<Texture> m_textureGround;
};
```

Ohne Typ-Alias

```
class TextureFactory
{
public:
    using TexturePtr = std::weak_ptr<Texture>;

    TexturePtr getOrCreateWallTexture();
    TexturePtr getOrCreateGroundTexture();
    // ...
private:
    TexturePtr m_textureWall;
    TexturePtr m_textureGround;
};
```

Mit Typ-Alias

1 Änderung
mit Alias

4 Änderungen
ohne Alias

Aliase

using & typedef



- Zur Definition von Typ-Aliassen existieren in C++ zwei Möglichkeiten
 - Mit dem Keyword „**typedef**“ wird zuerst der Datentyp angegeben, danach Aliasnamen, die Anstelle des Datentyps verwendet werden können; Leerzeichen getrennt
 - Syntax: `typedef Typname Aliasname[n];`
 - Beispiele:

<code>typedef unsigned int uint;</code>	<code>// Alias: uint</code>	<code>Auflösung: unsigned int</code>
<code>typedef const char* cstr;</code>	<code>// Alias: cstr</code>	<code>Auflösung: const char*</code>
 - **C++11** Mit dem Keyword „**using**“ wird zuerst ein Aliasname angegeben, der einem Datentypen direkt zugewiesen wird; Zuweisung mit „**=**“
 - Syntax: `using Aliasname = Typname;`
 - Beispiele:

<code>using uint = unsigned int;</code>	<code>// Alias: uint</code>	<code>Auflösung: unsigned int</code>
<code>using cstr = const char*;</code>	<code>// Alias: cstr</code>	<code>Auflösung: const char*</code>

Achtung: Das **using** Keyword hat im Zusammenhang mit Namensräumen eine andere Bedeutung!
Mit „**using namespace**“ werden Symbole eines Namensraums in den aktuellen Geltungsbereich (Scope) eingebunden

Aliase



using & typedef – Übersichtlichkeit

- Bei **typedef** Definitionen ist die **Abgrenzung** zwischen dem „Quell“-Typen und dem Aliasnamen nicht immer einfach ersichtlich

Beispiel: Mehrere Aliasnamen

```
typedef Texture* TexPtr, TexPtr2, &&TexRRef;
```

- TexPtr ist der Alias für einen Pointer auf ein Texture Objekt
- TexPtr2 ist der Alias für ein Texture Objekt (kein Pointer!)
- TexRRef ist der Alias auf eine R-Value-Reference auf ein Texture Objekt (kein Pointer!)

→ Undurchsichtige Definition von Kennzeichnern

Beispiel: Funktionspointer

```
typedef void (*fp)(int, const std::string&);
```

fp ist der Alias für eine Funktion, die als Parameter einen int, eine L-Value-Reference auf einen std::string erhält und nichts zurückgibt (void)

→ Alias in Funktionssignatur versteckt
→ Schwierig zu lesen

Aliase



using & typedef – Übersichtlichkeit

- Bei **using** Definitionen wird ein Aliasname eindeutig einem „Quell“-Typen zugewiesen
 - klare Abgrenzung von Alias und Quelltyp
 - klare Definition von Kennzeichnern

```
using TexPtr = Texture*;  
using TexPtr2 = Texture*;  
using TexPtrRRef = Texture*&&;
```

statt: `typedef Texture *TexPtr, *TexPtr2, *&&TexPtrRRef;`

- Bei Funktionspointern steht der Alias nicht mehr in der Funktionssignatur
 - klare Abgrenzung von Parametern & Rückgabetyt

```
using fp = void (*)(int, const std::string&);
```

statt: `typedef void (*fp)(int, const std::string&);`



Aliase

using & typedef – Templates

- Bei **typedef** Definitionen können variable Template-Argumente nicht direkt verwendet werden

```
template <typename T>  
typedef std::shared_ptr<T> OwnedPtr; // Compilerfehler
```

- Es bedarf der Abhilfe über Hilfsstrukturen:

```
template <typename T>  
struct OwnedPtr // Hilfsstruktur  
{  
    typedef std::shared_ptr<T> type;  
};  
  
// Verwendung:  
OwnedPtr<Texture>::type texture;  
// = std::shared_ptr<Texture> texture;
```

Diese Art von Hilfsstrukturen findet man häufig in der `<type_traits>` Standardbibliothek. Beispiele sind:

- `std::is_same<T1, T2>::value`
- `std::is_base_of<TBase, TDerived>::value`
- `std::conditional<bool, T1, T2>::type`
- ...

Aliase



using & typedef – Templates

- Bei **using** Definitionen können variable Template-Argumente direkt verwendet werden



```
template <typename T>  
using OwnedPtr = std::shared_ptr<T>;
```

- Der Alias kann direkt (ohne Hilfsstrukturen) verwendet werden:

```
// Verwendung:  
OwnedPtr<Texture> texture;  
// = std::shared_ptr<Texture> texture;
```

Alias-Definitionen der Standardbibliothek, die über Hilfsstrukturen definiert wurden, sind ab **C++14** auch über direkte Aliase verfügbar:

- `std::is_same_v<T1, T2>` (`_v` statt `::value`)
- `std::is_base_of_v<TBase, TDerived>` (`_v` statt `::value`)
- `std::conditional_t<bool, T1, T2>` (`_t` statt `::type`)
- ...



Aliase

Zusammenfassung

- Die Definition von **Typ-Aliasen** ist sinnvoll, wenn Typnamen lange oder verschachtelt sind und/oder häufig wiederholt werden
- Alias Deklarationen mit **using** sollten veralteten Definitionen mit **typedef** vorgezogen werden, da ...
 - eine klare **Abgrenzung** von Quelltyp und Aliasname stattfindet,
 - Kennzeichner **eindeutig zugeordnet** werden können,
 - **variable Template-Argumente** verwendet werden können

Aliase

Verständnisfragen

PIN: 774170

<https://poll.hs-kempten.de/poll/>



- **Was sind Vorteile von Typ-Aliasen?**
 - a) Sie verbessern die Wartbarkeit eines Projekts.
 - b) Sie führen zu einer besseren Performanz zur Laufzeit.
 - c) Sie ermöglichen implizite Umwandlungen zwischen Datentypen.
 - d) Sie verbessern die Lesbarkeit von langen Typnamen.
- **Warum sollten Typ-Aliase mit „using“ statt „typedef“ deklariert werden?**
 - a) **using** Definitionen sind schneller als **typedef** Definitionen.
 - b) **using** Definitionen sind besser lesbar als **typedef** Definitionen.
 - c) **using** Definitionen sind weniger fehleranfällig als **typedef** Definitionen.
 - d) **using** Definitionen können variable Template-Argumente direkt verwenden.



Compilerattribute

Lernziele

- Die Syntax und Funktionsweise von **Compilerattributen** wurde verstanden
- Compilerattribute können in eigenen Projekten unterstützend genutzt werden, um das Fehlerpotential zu verringern und die Lesbarkeit zu verbessern

Compilerattribute



Compiler-spezifische Attribute

- Mit **Compilerattributen** können dem Compiler **weiterführende Hinweise** gegeben werden, die über die Möglichkeiten der Syntax der Sprache hinausgehen
- I.d.R. definieren Compiler ihre eigenen Attribute, die nur für den jeweiligen Compiler funktionieren, z.B.:
 - `__declspec(attribute)` bei MSVC (Microsoft)
 - `__attribute__((attribute))` bei GNU und IBM
 - ...
- Beispiel: Definition einer Klasse „Texture“ in einer Bibliothek, die so kompiliert werden soll, dass Texture außerhalb der Bibliothek genutzt werden kann:

```
__declspec(dllexport) class Texture { /*...*/ };  
// dllexport: Die Klasse "Texture" wird in eine  
// Dynamic-link library (DLL) exportiert
```

MSVC

```
__attribute__((visibility("default"))) class Texture { /*...*/ };  
// visibility("default"): Die Klasse "Texture" ist außerhalb  
// der geteilten Bibliothek sichtbar
```

GNU / IBM

Compilerattribute



Standardisierte Attribute

- Mit dem **C++11** Standard wurden **allgemeingültige Compilerattribute** eingeführt, die auf jedem (C++11-konformen) Compiler funktionieren
 - Syntaktisch werden Compilerattribute des Standards in einer doppelten eckigen Klammer definiert:

```
[[attribute-list]]
```

- Beispiele:

```
[[maybe_unused]] int mightBeUnused;    // Variable wird u.U. nicht verwendet
[[deprecated]] void myOldFunction();     // Funktion ist veraltet
[[noreturn]] void terminateProgramm();   // Funktion wird nach Aufruf nicht mehr verlassen
```

- In Deklarationen können Compilerattribute an beliebiger Stelle stehen

```
[[maybe_unused]] int mightBeUnused;    // Ok
int [[maybe_unused]] mightBeUnused;    // Ok
int mightBeUnused [[maybe_unused]];    // Ok
```


Compilerattribute

Standardisierte Attribute



- Standardisierte Compilerattribute helfen häufig bei der Verwaltung von Warnungsmeldungen, die vom Compiler ausgelöst werden
- Das dient vor allem ...
 - der Vermeidung von Programmierfehlern durch die Auslösung von Compilerwarnungen, wo sonst keine entstehen würden, aber auch ...
 - der Deaktivierung von Compilerwarnungen, wo diese störend und im Kontext der Anweisung fehl am Platz sind
- Nachfolgend werden die wichtigsten dieser Compilerattribute kurz aufgezeigt

Compilerattribute



nodiscard

- Das `[[nodiscard]]` Attribut (seit `C++17`) gibt an, dass der **Rückgabewert** einer Funktion nicht verworfen werden sollte
 - Wird der Rückgabewert dennoch verworfen, erzeugt der Compiler eine entsprechende Warnmeldung
- Beispiel: Funktion, die eine Datei von der Festplatte einliest:

```
std::vector<uint8_t> readFile(const std::string& filePath)
{
    std::vector<uint8_t> readBytes;

    std::ifstream ifs(filePath, std::ios::binary);
    // Lesen aller Bytes aus Datei in den Vector "readBytes" ...

    return readBytes;
}
```

Compilerattribute



nodiscard – Beispiel

- Beispiel: Der Rückgabewert der Funktion `readFile(filePath)` – also die Binärdaten der Datei – sollten abgespeichert oder anderweitig weiterverwendet werden:

```
std::vector<uint8_t> readFile(const std::string& filePath)
{/* ... */}
```

```
int main()
{
    readFile("wall.png");
}
```

Funktion wird aufgerufen, ohne dass
der Rückgabewert verwendet wird
→ valider Funktionsaufruf
→ Daten gehen verloren
→ Mehraufwand für nichts

Hier entsteht keine Warn- oder Fehlermeldung!
→ Der Programmierfehler geht unentdeckt unter



Compilerattribute

nodiscard – Beispiel

- Beispiel: Der Funktion wird nun das `[[nodiscard]]` Attribut angehängt:

Compilerattribut

```
[[nodiscard]] std::vector<uint8_t> readFile(const std::string& filePath)
{/* ... */}
```

```
int main()
{
    readFile("wall.png");
}
```

Beim Aufruf ohne Behandlung des Rückgabewerts entsteht nun eine Compilerwarnung

 C4834 discarding return value of function with `[[nodiscard]]` attribute

→ Der Compiler unterstützt durch Erzeugung der Warnmeldung beim Auffinden des Programmierfehlers

Compilerattribute

nodiscard – Beispiel



- Das `[[nodiscard]]` Attribut hilft auch bei der Definition und Lesbarkeit von **Schnittstellenfunktionen**
- Beispiel: Funktionen einer (eigenen) Containerklasse

Rückgabewert ist nur eine **Zusatzinformation** und kann ignoriert werden
→ discardable (verwerfbar)

Rückgabe enthält die **Hauptinformation**/-aufgabe der Funktion und sollte verarbeitet werden
→ `nodiscard`

```
class Vector
{
public:
    // Löscht alle Elemente aus dem Container
    // Rückgabe: true, wenn Elemente gelöscht wurden, sonst false
    bool clear();

    // Abfrage, ob der Container leer ist
    // Rückgabe: true, wenn keine Elemente vorhanden, sonst false
    [[nodiscard]] bool isEmpty() const;
};
```

→ Bei Getter-Funktionen sollte immer `[[nodiscard]]` spezifiziert werden

Compilerattribute

fallthrough



- Ein häufiger Programmierfehler bei der Verwendung von **switch-case**-Anweisungen ist das Vergessen der „break“ Anweisung
- Das führt zu Programmierfehlern, bei denen nach Abhandlung eines Falls (cases) ungewollt in den nächsten „gefallen“ wird (= fallthrough)
- Beispiel: Vergessene **break** Anweisung

```
void printValue(int x)
{
    switch (x)
    {
        case 1:
            std::cout << "Wert ist 1" << std::endl;
        case 2:
            std::cout << "Wert ist 2" << std::endl;
            break;
    }
}
```

Vergessenes **break;**

case fällt durch (fallthrough)

```
int main()
{
    printValue(1);
}
```

```
Wert ist 1
Wert ist 2
```

Fehlerhafte Ausgabe

Compilerattribute



fallthrough

- Viele moderne Compiler erzeugen automatisch eine Warnmeldung, wenn bei einem **case** die **break** Anweisung fehlt

```
void printValue(int x)
{
    switch (x)
    {
        case 1:
            std::cout << "Wert ist 1" << std::endl;
        case 2:
            std::cout << "Wert ist 2" << std::endl;
            break;
    }
}
```

C26819: Unannotated fallthrough between switch labels (es.78).

Compilerattribute

fallthrough

- In manchen Fällen ist das „Durchfallen“ nach einem **case** jedoch gewollt
- Das **[[fallthrough]]** Attribut (seit **C++17**) gibt an, dass aus einem **case** gewollt in den nächsten durchgefallen wird
 - Der Compiler erzeugt dann keine entsprechende Warnmeldung mehr

Beispiel: Mehrere **cases** für die Behandlung von groß- und kleingeschriebenen Buchstaben:



```
void processInput()
{
    char input { 'n' };
    std::cout << "Operation durchfuehren? [j/n]: ";
    std::cin >> input;

    switch (input)
    {
        case 'j': [[fallthrough]];
        case 'J':
            continueOperation();
            break;

        case 'n': [[fallthrough]];
        case 'N': [[fallthrough]];
        default:
            abortOperation();
            break;
    }
}
```

[[fallthrough]] weist den Compiler auf das gewollte Durchfallen hin, wodurch keine Compilerwarnung erzeugt wird

Compilerattribute

Warnungsattribute



- Es existieren weitere standardisierte Compilerattribute, die Warnungen auslösen oder verhindern
- Übersicht:

„Warnungs“-Attribute		
<code>[[deprecated]]</code> <code>[[deprecated("Reason")]]</code>	Die Deklaration ist veraltet und sollte nicht mehr genutzt werden → Compilerwarnung bei Nutzung	C++14
<code>[[fallthrough]]</code>	Das „Durchfallen“ vom aktuellen case -Label in das nächste ist gewollt → Keine Compilerwarnung bei fehlendem break	C++17
<code>[[nodiscard]]</code> <code>[[nodiscard("Reason")]]</code>	Der Funktionsrückgabewert soll nicht verworfen werden → Compilerwarnung wenn Rückgabewert nicht verwendet wird	C++17 C++20
<code>[[maybe_unused]]</code>	Die Deklaration wird u.U. nicht verwendet → Keine Compilerwarnung, dass die Deklaration nicht verwendet wird und damit entfernt werden könnte	C++17

Compilerattribute



Optimierungsattribute

- Andere standardisierte Compilerattribute helfen bei der **Optimierung** des Maschinencodes durch Merkmale, die vom Compiler sonst nicht festgestellt werden können
- Diese führen über den Rahmen dieser Veranstaltung hinaus
- Übersicht:

„Optimierungs“-Attribute		
<code>[[noreturn]]</code>	Nach Aufruf der Funktion erfolgt keine Rückgabe (des Instruction-Pointers) in die aufrufende Funktion	C++11
<code>[[carries_dependency]]</code>	Die markierten Daten hängen von anderen Daten ab und können in Bezug auf Multithreading optimiert werden	C++11
<code>[[likely]]</code> <code>[[unlikely]]</code>	Der Programmzweig kommt häufiger oder weniger häufig vor als andere Zweige und kann dahingehend optimiert werden	C++20
<code>[[no_unique_address]]</code>	Die nicht-statische Membervariable benötigt keine einzigartige Speicheradresse im Layout der Datenstruktur	C++20
<code>[[assume(expression)]]</code>	Der Ausdruck „ <i>expression</i> “ wird immer zu true ausgewertet werden	C++23

Compilerattribute



Zusammenfassung

- Compilerattribute geben dem Compiler **weiterführende Hinweise** zur Verwendung einer Definition
- Es wird unterschieden zwischen
 - **Compiler-spezifischen** Attributen: Gelten nur für den jeweiligen Compiler
 - **Standardisierten** Attributen: Gelten für alle Compiler, die den jeweiligen C++-Standard unterstützen
- Viele standardisierte Attribute unterstützen den Programmierer bei der **Vermeidung von Programmierfehlern durch Erzeugung von Warnmeldungen**
 - Das Compilerattribut `[[nodiscard]]` hilft, das Vergessen von Returnwerten aus Funktionen zu vermeiden
- Andere standardisierte Attribute haben einen sehr speziellen Verwendungszweck und werden meist nicht benötigt

Compilerattribute

Verständnisfragen

PIN: 774170

<https://poll.hs-kempten.de/poll/>



- **Wozu werden Compilerattribute verwendet?**
 - a) Um dem Compiler zusätzliche Hinweise zur Optimierung oder Analyse des Codes zu geben.
 - b) Um bestimmte Warnungen zu unterdrücken oder zu erzwingen.
 - c) Um Fehler im Code automatisch zu korrigieren.
 - d) Um eine Dokumentation des Codes zu generieren.
- **Welches der folgenden Attribute wird verwendet, um zu signalisieren, dass der Rückgabewert einer Funktion gespeichert werden sollte?**
 - a) `[[return]]`
 - b) `[[discard]]`
 - c) `[[nodiscard]]`
 - d) `[[noignore]]`



Kompilierzeitkonstanten

Lernziele

- Die Unterscheidung von **Kompilierzeit** und **Laufzeit** kann nachvollzogen werden
- **Kompilierzeitkonstanten** können in eigenen Programmen eingesetzt werden
 - Hierzu wurde die Bedeutung und der Kontext des Keywords **constexpr** verstanden, sodass dieses anstelle von veralteten Definitionen eingesetzt werden kann

Kompilierzeitkonstanten



Konstanten

- Neben Laufzeitkonstanten werden bei der Programmierung häufig Konstanten verwendet, die sich über den gesamten Programmablauf nicht verändern und von Beginn des Programms an feststehen – diese werden **Kompilierzeitkonstanten** (compile-time constants) genannt
 - **Kompilierzeit:** Alle Operationen *vor* Programmstart
 - **Laufzeit:** Alle Operationen *nach* Programmstart / *während* Programmablauf
- Typische Kompilierzeitkonstanten sind ...
 - **numerische Konstanten**, wie die Zahl π oder die Eulersche Zahl(e) ...
 - aber auch **erweiterte Konstrukte und Objekte**, wie Strings
- Kompilierzeitkonstanten belegen i.d.R. keinen Speicher, bis sie von konkretem Programmcode verwendet werden → **PR-Values**
 - Nur bei komplexeren Konstrukten bildet der Compiler teilweise Objekte im Datenbereich des Prozessspeichers ab (für Programmierer nicht relevant)

Kompilierzeitkonstanten



Präprozessor-Definitionen (veraltet)

- Vor dem **C++11** Standard wurden Kompilierzeitkonstanten über **Präprozessor-Definitionen** definiert
 - Da es keine **syntaktische** Möglichkeit gab, Kompilierzeitkonstanten zu definieren

```
#define PI 3.14159265358f
```

- Präprozessor-Definitionen werden lediglich durch Text im Programmcode ersetzt, was zu Problemen bei Kompilierzeitkonstanten führt ...
 - Fehlende Typ-Sicherheit
 - Keine Anerkennung von Scopes und Namespaces
 - Keine Berechnungen zur Kompilierzeit
 - Andere Probleme mit dem Präprozessor (Makro-Expansion, Doppelnennungen, ...)

Kompilierzeitkonstanten



constexpr – Variablen

- Zur Lösung der Defizite von Präprozessor-Definitionen, führt der **C++11** Standard das Keyword „**constexpr**“ (= constant expression = konstanter Ausdruck) für Kompilierzeitkonstanten ein
- Wird das **constexpr** Keyword auf eine **Variable** angewendet, wird die Variable als Kompilierzeitkonstante definiert
 - **constexpr** kann auch auf **Funktionen** angewendet werden, hat dann aber eine andere Bedeutung
 - Zunächst wird nur die Anwendung auf Variablen betrachtet

```
// C++11
constexpr float PI { 3.14159265358f };

// vor C++11
#define PI 3.14159265358f
```

Neu:

- Angabe des Datentyps
- Deklaration & Initialisierung als Variable
- **constexpr** Keyword

Kompilierzeitkonstanten



constexpr – Variablen – Bedingungen

- Variablen, die **constexpr** deklariert werden, müssen Bedingungen erfüllen
 - Die Variable muss sich **zur Kompilierzeit erstellen** und **zerstören** lassen – das ist der Fall bei:
 - Trivialen Datentypen (Skalare, Referenzen, Arrays)
 - Typen mit **constexpr** Konstruktoren und Destruktoren
 - Lambda-Objekten

→ Schließt vor allem dynamisch allokierende Objekte aus!
 - Die Variable **muss initialisiert werden** (keine reinen Deklarationen ohne Wert)
 - Der gesamte **Initialisierungsausdruck** der Variable muss zur **Kompilierzeit ausgewertet** werden können, das beinhaltet:
 - (Implizite) Umwandlungen
 - Konstruktoraufrufe
 - ...

→ Auch hier dürfen keine Allokationen entstehen!

Kompilierzeitkonstanten



Typ-Sicherheit

- `constexpr` deklarierte Kompilierzeitkonstanten haben einen festgesetzten Datentyp (wie bei Variablen üblich) = **Typ-sicher**

Präprozessor-Definition (veraltet)

```
int main()
{
    // Typ ist nicht definiert
    #define VAL 100

    int v1 { VAL };           // Ok
    long long v2 { VAL };    // Ok
    double v3 { VAL };       // Ok
}
```

`constexpr` (C++11)

```
int main()
{
    // Typ definiert als double
    constexpr double VAL = 100;

    int v1 { VAL };           // Compilerfehler
    long long v2 { VAL };    // Compilerfehler
    double v3 { VAL };       // Ok
}
```

Kompilierzeitkonstanten



Scopes

- `constexpr` deklarierte Kompilierzeitkonstanten **existieren nur im umschließenden Scope** (wie bei Variablen üblich)

Präprozessor-Definition (veraltet)

```
void printSinPi()
{
    #define PI 3.14159265358f

    std::cout << "Sine of Pi is: " << std::sin(PI);
}

int main()
{
    printSinPi();
    std::cout << PI; // Funktioniert
}
```

```
Sine of Pi is: 0
3.14159
```

`constexpr` (C++11)

```
void printSinPi()
{
    constexpr float PI { 3.14159265358f };

    std::cout << "Sine of Pi is: " << std::sin(PI);
}

int main()
{
    printSinPi();
    std::cout << PI; // Compilerfehler
}
```

Kompilierzeitkonstanten



Scopes

- **constexpr** deklarierte Kompilierzeitkonstanten **existieren nur im umschließenden Scope** (wie bei Variablen üblich)

Präprozessor-Definition (veraltet)

```
void printSinPi()
{
    #define PI 3.14159265358f

    std::cout << "Sine of Pi is: " << std::sin(PI);
}

int main()
{
    printSinPi();
    std::cout << PI; // Funktioniert
}
```

Präprozessor-Definitionen
existieren ab ihrer Definition
unabhängig von Scopes

```
Sine of Pi is: 0
3.14159
```

constexpr (C++11)

```
void printSinPi()
{
    constexpr float PI { 3.14159265358f };

    std::cout << "Sine of Pi is: " << std::sin(PI);
}

int main()
{
    printSinPi();
    std::cout << PI; // Compilerfehler
}
```

(constexpr) Variablen
existieren nur im
umschließenden Scope

Kompilierzeitkonstanten



Scopes

- Die Vermeidung von Präprozessor-Definitionen trägt erheblich zur Sauberkeit eines Programms bzw. einer Bibliothek bei
 - Da der **globale Namensraum** (global namespace) nicht mit Definitionen verschmutzt wird
- Gerade in Header-Dateien sind Präprozessor-Definitionen problematisch

```
namespace Physics
{
    #define PI 3.14159265358f

    class RigidBody
    {
        void applyForce(float angle, float strength);
        ...
    };
}
```

PhysicsApi.h

```
namespace Graphics
{
    #define PI 3.14159265358f

    class Transform
    {
        void rotate(float angle);
        ...
    };
}
```

GraphicsApi.h

```
#include "PhysicsApi.h"
#include "GraphicsApi.h"

int main()
{
    Physics::RigidBody rb;
    Graphics::Transform t;
    ...
}
```

main.cpp

Kompilierzeitkonstanten

Scopes



Re-Definition von PI durch
verschiedene Header

```
namespace Physics
{
    #define PI 3.14159265358f

    class RigidBody
    {
        void applyForce(float angle, float strength);
        ...
    };
}
```

PhysicsApi.h

Bibliothek 1

```
namespace Graphics
{
    #define PI 3.14159265358f

    class Transform
    {
        void rotate(float angle);
        ...
    };
}
```

GraphicsApi.h

Bibliothek 2

```
#include "PhysicsApi.h"
#include "GraphicsApi.h"

int main()
{
    Physics::RigidBody rb;
    Graphics::Transform t;
    ...
}
```

main.cpp

Nutzendes Programm

→ Mindestens nervige Compilerwarnung (wegen Redefinition)

Kompilierzeitkonstanten

Berechnungen



- **constexpr** Kompilierzeitkonstanten können durch **Berechnungen** zugewiesen werden bzw. aus diesen resultieren
 - Dabei berechnet der Compiler den Wert vorab, sodass die **constexpr** Variable lediglich das Ergebnis der Berechnung enthält

Präprozessor-Definition (veraltet)

```
#define PI 3.14159265358f
#define PI_SQUARED (PI * PI)

int main()
{
    std::cout << "Pi squared is: " << PI_SQUARED;
}
```

constexpr (C++11)

```
constexpr float PI { 3.14159265358f };
constexpr float PI_SQUARED { (PI * PI) };

int main()
{
    std::cout << "Pi squared is: " << PI_SQUARED;
}
```

Kompilierzeitkonstanten

Berechnungen – Beispiel



- Präprozessor-Definitionen werden lediglich **expandiert**, nicht vorberechnet

Präprozessor-Definition (veraltet)

```
#define PI 3.14159265358f
#define PI_SQUARED (PI * PI)

int main()
{
    std::cout << "Pi squared is: "
               << PI_SQUARED;
}
```

Expansion 1

```
#define PI 3.14159265358f

int main()
{
    std::cout << "Pi squared is: "
               << (PI * PI);
}
```

Expansion 2

```
int main()
{
    std::cout << "Pi squared is: "
               << (3.14..f * 3.14..f);
}
```

ASM (vereinfacht)

```
main:
; Setze die Fließkommazahl 3.14 in das Register xmm0
movss xmm0, 3.14..f

; Multipliziere 3.14f mit sich selbst in xmm0
mulss xmm0, xmm0

; Rufe die Ausgabe-Funktion auf
call cout, "Pi squared is: ", xmm0
```

Kompilierung



→ Berechnung erfolgt zur **Laufzeit**

Kompilierzeitkonstanten

Berechnungen – Beispiel



- **constexpr** Kompilierzeitkonstanten werden vom Compiler ***vorberechnet***

constexpr (C++11)

```
constexpr float PI { 3.14159265358f };  
constexpr float PI_SQUARED { (PI * PI) };  
  
int main()  
{  
    std::cout << "Pi squared is: " << PI_SQUARED;  
}
```

Kompilierung

ASM (vereinfacht)

```
main:  
; Setze das vorberechnet Ergebnis direkt in das Register xmm0  
movss xmm0, 9.87..f ; 3.14.. * 3.14.. = 9.87..  
  
; Multiplikation zur Laufzeit entfällt  
  
; Rufe die Ausgabe-Funktion auf  
call cout, "Pi squared is: ", xmm0
```

Compiler führt (PI * PI) aus



→ Berechnung erfolgt zur **Kompilierzeit**

Kompilierzeitkonstanten



Komplexe Konstrukte

- Auch komplexere Konstrukte können mit `constexpr` Kompilierzeitkonstanten dargestellt und berechnet werden

```
// Array-Kompilierzeitkonstante
constexpr const char* OPTIONS[]
{
    "Zurueck",
    "Ignorieren",
    "Weiter"
};

// Berechnung der Arraylänge zur Kompilierzeit
constexpr size_t OPTION_COUNT {
    sizeof(OPTIONS) / sizeof(*OPTIONS);
};
```

```
int main()
{
    // Nutzung der Kompilierzeitkonstanten im Code
    for (size_t i = 0u; i < OPTION_COUNT; i++)
    {
        std::cout << "Option " << i
            << " ist: " << OPTIONS[i]
            << std::endl;
    }
}
```

```
Option 0 ist: Zurueck
Option 1 ist: Ignorieren
Option 2 ist: Weiter
```



Kompilierzeitfunktionen

Lernziele

- Der Nutzen und die Eigenschaften von Kompilierzeit**funktionen** wurde verstanden
- Kompilierzeitfunktionen können in eigenen Programmen definiert werden
 - Hierzu wurde die Bedeutung des **constexpr** Keywords angewandt auf **Funktionen** verstanden

Kompilierzeitfunktionen



constexpr Funktionen

- Wird das „**constexpr**“ Keyword auf Funktionen angewendet, wird eine Funktion definiert, die vom Compiler ausgeführt werden kann = **Kompilierzeitfunktion** C++11
 - Sie kann weiterhin regulär zur Laufzeit aufgerufen werden – wie andere Funktionen
- Eine **constexpr** Funktion kann als Kompilierzeitkonstante verwendet werden

```
// vom Compiler ausführbare Funktion
constexpr float square(float x)
{
    return x * x;
}

constexpr float PI { 3.14159265358f };

// zur Kompilierzeit berechnet
constexpr float PI_SQUARED { square(PI) };
```

```
int main()
{
    // Nutzung vorcompilierter Konstantenwert
    std::cout << "Pi squared is: " << PI_SQUARED; // Konstante
    std::cout << "Pi squared is: " << square(PI); // Konstante

    // Berechnung zur Laufzeit
    float input { 0.0f };
    std::cout << "Enter number: ";
    std::cin >> input;
    std::cout << "Number squared is: " << square(input); // Laufzeitvariable
}
```

Kompilierzeitfunktionen

Bedingungen



- Funktionen, die **constexpr** deklariert werden, müssen Bedingungen erfüllen
 - Die verwendeten Variablen – inklusive Parameter und Rückgabewert – müssen die Bedingungen von **constexpr** Variablen erfüllen:
 - Erstellung & Zerstörung zur Kompilierzeit,
 - Initialisierung notwendig,
 - Initialisierung zur Kompilierzeit auswertbar
 - Außerdem dürfen keine statischen oder Thread-lokalen Variablen verwendet werden
 - In der Funktion dürfen bestimmte „Sprung“-Anweisungen nicht verwendet werden, wie:
 - **goto** Anweisungen (und goto-Labels)
 - Exceptions (**throw**, **try**, **catch**)
 - Aufruf virtueller Funktionen
 - Die Funktion kann nicht separat von ihrer Deklaration definiert werden (z.B. in .cpp-Datei)

Bis zu den **C++14** und **C++20** Standards gab es weitere Einschränkungen für **constexpr** Funktionen, z.B. dass keine lokalen Variablen, Zweige und Schleifen zulässig waren. Bei der Verwendung ist auf den C++-Standard des jeweiligen Compilers zu achten!

Kompilierzeitfunktionen



Beispiel

- Beispiel: Es soll eine, vom Compiler ausführbare, Funktion geschrieben werden, die die größte Zahl in einem (möglicherweise konstanten) Array von Zahlen ermittelt

NumberCount wird zur Arraylänge des übergebenen Kompilierzeitarrays *deduziert*

```
template <size_t NumberCount>
constexpr int getHighestNumber(const int (&numbers)[NumberCount])
{
    int highest { numbers[0] };

    for (size_t i = 1u; i < NumberCount; ++i)
    {
        if (numbers[i] > highest)
        {
            highest = numbers[i];
        }
    }

    return highest;
}
```

Kompilierzeitfunktionen

Beispiel



```
template <size_t NumberCount>
constexpr int getHighestNumber(const int (&numbers)[NumberCount])
{
    int highest { numbers[0] };
    for (size_t i = 1u; i < NumberCount; ++i)
    {
        if (numbers[i] > highest)
        {
            highest = numbers[i];
        }
    }

    return highest;
}
```

Seit C++14 möglich:

- Lokale Variablen
- Schleife (for)
- Zweige (if)

Vom Compiler berechenbar:

```
// Array-Kompilierzeitkonstante
constexpr int NUMBERS[] {
    1, 3, 2, 8, 21, 13, 5
};

// Compiler-berechnet
constexpr int HIGHEST_NUMBER {
    getHighestNumber(NUMBERS);
};

int main()
{
    // Ausgabe als Konstante
    std::cout << "The highest number is: "
                << HIGHEST_NUMBER;
}
```

The highest number is 21

Kompilierzeitfunktionen



constexpr

- Seit **C++20** ist es mit dem Keyword „**constexpr**“ möglich festzulegen, dass eine Funktion zur Kompilierzeit ausgewertet werden *muss*
 - Das garantiert, dass durch die Funktion kein Berechnungsaufwand zur Laufzeit entstehen kann
 - Das **constexpr** Keyword erlaubt dagegen auch die Auswertung zur Laufzeit

```
// muss vom Compiler ausgeführt werden
constexpr float square(float x)
{
    return x * x;
}

constexpr float PI { 3.14159265358f };

// Ok: vom Compiler ausgeführt, da PI konstant
constexpr float PI_SQUARED { square(PI) };
```

```
int main()
{
    std::cout << "Pi squared is: " << PI_SQUARED; // Ok: Konstante
    std::cout << "Pi squared is: " << square(PI); // Ok: Konstante

    float input { 0.0f };
    std::cout << "Enter number: ";
    std::cin >> input;
    std::cout << "Number squared is: "
                << square(input); // Compilerfehler
}
```

Auswertung würde zur Laufzeit erfolgen



Kompilierzeitklassen

Lernziele

- Der Nutzen und die Eigenschaften von Kompilierzeit**klassen** wurde verstanden
- Kompilierzeitklassen können in eigenen Programmen definiert werden
 - Hierzu wurde die Bedeutung des **constexpr** Keywords *angewandt auf Konstruktoren und Destruktoren* verstanden

Kompilierzeitklassen



constexpr Klassen

- Durch die Anwendung des „**constexpr**“ Keywords auf **Methoden** einer Klasse, können Instanzen bzw. **Objekte als Kompilierzeitkonstanten** verwendet werden **C++11**
 - D.h. entweder als Konstanten selbst oder als Variablen in einer **constexpr** Funktion
- Entscheidend ist dabei die Deklaration des **Konstruktors** und (seit **C++20**) des **Destruktors** als **constexpr** Methoden; Impliziter Destruktor ist **constexpr**, falls nicht angegeben.

```
class ConstexprObject
{
public:
    // Fehler: nicht constexpr
    ConstexprObject(int x)
        : m_x{x} {}

private:
    int m_x;
};
```

```
class ConstexprObject
{
public:
    // Ok
    constexpr ConstexprObject(int x)
        : m_x{x} {}

private:
    int m_x;
};
```

```
class ConstexprObject
{
public:
    constexpr ConstexprObject(int x)
        : m_x{x} {}
    ~ConstexprObject() {} // Fehler: nicht constexpr

private:
    int m_x;
};
```

```
constexpr ConstexprObject OBJ { 42 };
```

Kompilierzeitklassen

Bedingungen



- Klassen selbst können nicht **constexpr** deklariert werden
 - Der „Konstanten-Status“ liegt auf den einzelnen Methoden
- **constexpr** Methoden einer Klasse – inklusive des Konstruktors und Destruktors – sind Kompilierzeitfunktionen und unterliegen damit deren Bedingungen:
 - Verwendete Variablen müssen die Bedingungen für **constexpr** Variablen erfüllen (= zur Kompilierzeit auswertbar sein)
 - Bestimmte Anweisungen dürfen nicht verwendet werden (goto, Exceptions, virtual, ...)
- Für den **constexpr Konstruktor** gilt außerdem, dass er jede nicht-statische Membervariable initialisieren muss
→ **constexpr** Konstruktoren garantieren vollständig initialisierte Objekte!

Bis zum **C++ 20** Standard war es nicht möglich **constexpr** Destruktoren zu definieren. Auch die Standard-abhängigen Einschränkungen auf **constexpr** Funktionen treffen auf **constexpr** Konstruktoren und Destruktoren zu. Bei der Verwendung ist auf den C++-Standard des jeweiligen Compilers zu achten!

Kompilierzeitklassen



Beispiel

- Beispiel: Es soll eine Array-Klasse für Zahlen geschrieben werden, die zur Kompilierzeit als Konstante verwendet werden kann

Gewünschte Verwendung:

```
// Array-Kompilierzeitkonstante
constexpr ConstexprIntArray<7u> NUMBERS
{
    1, 3, 2, 8, 21, 13, 5
};

// Methodenaufruf zur Kompilierzeit
constexpr size_t NUMBER_COUNT { NUMBERS.getSize() };

// Operatoraufruf zur Kompilierzeit
constexpr int NUMBER_AT_3 { NUMBERS[3] };
```

Kompilierzeitklassen



Beispiel

- Es wird eine Klasse „**constexprIntArray**“ mit **constexpr** Konstruktor und Methoden definiert:

constexpr Klasse durch Konstruktor:

```
template <size_t ElementCount>
class constexprIntArray
{
public:
    constexpr constexprIntArray(std::initializer_list<int> elements)
    {
        for (size_t i = 0u; i < ElementCount; i++)
            m_elements[i] = *(elements.begin() + i);
    }

private:
    int m_elements[ElementCount];
};
```

constexpr Methoden:

```
template <size_t ElementCount>
class constexprIntArray
{
    constexpr size_t getSize() const
    {
        return ElementCount;
    }

    constexpr int operator[](size_t index) const
    {
        return m_elements[index];
    }
    ...
};
```

Kompilierzeitklassen

Beispiel



- Die Klasse kann vollständig zur Kompilierzeit verwendet werden:

```
// Array-Kompilierzeitkonstante
constexpr ConstexprIntArray<7u> NUMBERS
{
    1, 3, 2, 8, 21, 13, 5
};

// Methodenaufruf zur Kompilierzeit
constexpr size_t NUMBER_COUNT { NUMBERS.getSize() };

// Operatoraufruf zur Kompilierzeit
constexpr int NUMBER_AT_3 { NUMBERS[3] };

int main()
{
    std::cout << "Number count is: " << NUMBER_COUNT;
    std::cout << "Number at index 3 is: " << NUMBER_AT_3;
}
```

Aufruf der Methoden
von `ConstexprIntArray`
zur Kompilierzeit

Ausgabe:

```
Number count is: 7
Number at index 3 is: 8
```

Kompilierzeitklassen

Beispiel



- Die `constexpr` Methoden der Klasse können auch zur Laufzeit verwendet werden:

```
// Array-Kompilierzeitkonstante
constexpr ConstexprIntArray<7u> NUMBERS { 1, 3, 2, 8, 21, 13, 5 };

int main()
{
    for (size_t i = 0u; i < NUMBERS.getSize(); i++)
    {
        std::cout << "Number " << i
                  << " is: " << NUMBERS[i]
                  << std::endl;
    }
}
```

Ausgabe:

```
Number 0 is: 1
Number 1 is: 3
Number 2 is: 2
Number 3 is: 8
Number 4 is: 21
Number 5 is: 13
Number 6 is: 5
```

Kompilierzeitklassen

Standardbibliothek



- Einige Klassen der Standardbibliothek wurden für die Verwendung als Kompilierzeitkonstanten ertüchtigt und können **constexpr** verwendet werden, u.a.:

C++11

std::array (ähnlich dem `constexprIntArray` Beispiel)

```
// Array-Kompilierzeitkonstante
constexpr std::array<int, 7u> NUMBERS =
{
    1, 3, 2, 8, 21, 13, 5
};

// Methodenaufruf zur Kompilierzeit
constexpr size_t NUMBER_COUNT = NUMBERS.size();

// Operatoraufruf zur Kompilierzeit
constexpr int NUMBER_AT_3 = NUMBERS[3];
```

C++17

std::string_view für Zeichenketten

```
// String-Kompilierzeitkonstante
constexpr std::string_view TEXT = "Hallo Kempten";

// Methodenaufruf zur Kompilierzeit
constexpr size_t TEXT_SIZE = TEXT.size();

// Übliche String-Methoden zur Kompilierzeit
constexpr char TEXT_LAST_CHAR = *TEXT.rbegin();
```


Kompilierzeitkonstanten, -funktionen und -klassen



Zusammenfassung

- Das **constexpr** Keyword, angewandt auf **Variablen**, definiert Konstanten, die zur Kompilierzeit bekannt sind
- Das **constexpr** Keyword, angewandt auf **Funktionen**, definiert, dass eine Funktion zur Kompilierzeit ausgewertet werden *kann*
 - Werden **constexpr** Funktionen mit Laufzeitvariablen aufgerufen, fungieren sie als normale Funktionen und werden ebenfalls zur Laufzeit aufgerufen
 - Das **constexpr** Keyword schränkt weiter ein, dass eine Funktion zur Kompilierzeit ausgewertet werden *muss*
- Mit **constexpr** **Konstruktoren** und **Destruktoren** wird definiert, dass Instanzen einer **Klasse** als Kompilierzeitkonstanten verwendet werden können
 - Weiter garantieren **constexpr** Konstruktoren, dass ein Objekt vollständig initialisiert wird
- Das **constexpr** Keyword sollte so viel wie möglich eingesetzt werden [Meyers14]

[Meyers14] : Scott Meyers, Effective Modern C++, 2014

Kompilierzeitkonstanten, -funktionen und -klassen

Verständnisfragen



■ Welche Aussage beschreibt Kompilierzeitkonstanten am besten?

- a) Objekte, die vor Programmstart initialisiert werden.
- b) Objekte, die sich während der Ausführung eines Programms nicht verändern.
- c) Objekte, die keinen Speicherplatz belegen.
- d) Objekte, die numerische Konstanten darstellen.

■ Welche Aussage beschreibt Kompilierzeitfunktionen am besten?

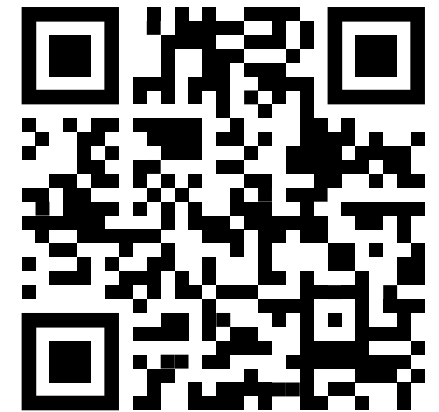
- a) Funktionen, die vor Programmstart definiert werden.
- b) Funktionen, die vor Programmstart ausgeführt werden können.
- c) Funktionen, die nur konstante Parameter verwenden können.
- d) Funktionen, die keine Exceptions werfen.

PIN: 774170

<https://poll.hs-kempten.de/poll/>

■ Welche Aussage beschreibt Kompilierzeitklassen am besten?

- a) Klassen, die vor Programmstart definiert werden.
- b) Klassen, die mindestens eine constexpr-Methode besitzen.
- c) Klassen, die nur konstante Member besitzen.
- d) Klassen, deren Instanzen vor Programmstart initialisiert werden können.



Kompilierzeitfunktionen

Verständnisfragen



■ WAHR oder FALSCH?

- Kompilierzeitkonstanten sind PR-Values.
- Kompilierzeitkonstanten können zur Laufzeit verändert werden.
- constexpr-deklarierte Variablen müssen initialisiert werden.
- Die Initialisierung einer constexpr-deklarierte Variable darf Laufzeitvariablen enthalten.
- Eine constexpr-deklarierte Funktion muss vom Compiler ausgeführt werden.
- Eine constexpr-deklarierte Funktion kann lokale Variablen dynamisch allokalieren.
- Kompilierzeitklassen werden durch Anhängen des Keywords „constexpr“ vor „class“ definiert.
- Der Konstruktor einer Kompilierzeitklasse ist eine Kompilierzeitfunktion.
- Instanzen von Kompilierzeitklassen sind immer vollständig initialisiert.
- Alle Methoden einer Kompilierzeitklasse sind automatisch constexpr-deklariert, wenn der Konstruktor constexpr deklariert wurde.



Kompilierzeitweige

Lernziele

- Der Unterschied der **Auswertung von Programmzweigen** zur Laufzeit vs. zur Kompilierzeit wurde verstanden
- Die **Vorteile** von Kompilierzeitzweigen können nachvollzogen werden

Kompilierzeitweig



Laufzeitweig vs. Kompilierzeitweig

- Verzweigungen (**if-else**, **switch-case**) werden üblicherweise zur Laufzeit ausgewertet
 - Meist über Sprung-Befehle im Maschinencode (**jmp**, **je**, **jz**, ...)
- Manche Verzweigungen können jedoch bereits zur Kompilierzeit ausgewertet werden
 - Beispielsweise Anweisungen auf Template-Argumenten / Kompilierzeitkonstanten
- Seit dem **C++17** Standard ist es möglich, das **constexpr** Keyword auf eine **if**-Anweisung anzuwenden und damit die Auswertung eines Zweiges zur Kompilierzeit zu erzwingen = **Kompilierzeitweig**
 - **constexpr if**-Zweige können nur auf Kompilierzeitkonstanten angewendet werden (Template-Argumente, **constexpr** Funktionen, ...)
 - Ist der **if**-Zweig **constexpr** deklariert, so ist der dazugehörige **else**-Zweig ebenfalls **constexpr**
 - **constexpr** kann nicht auf **switch-case**-Anweisungen angewendet werden

Kompilierzeitweig



Laufzeitweig vs. Kompilierzeitweig – Beispiel

- Beispiel: Eine Funktion zur Ausgabe, ob der Parameter eine gerade oder ungerade Zahl ist

```
void printEven(int x)
{
    if ((x % 2) == 0)
        std::cout << "X is even";
    else
        std::cout << "X is odd";
}

int main()
{
    printEven(11);
    printEven(42);
}
```

Laufzeitparameter

Check / Verzweigung zur Laufzeit

```
template <int x>
void printEven()
{
    if ((x % 2) == 0)
        std::cout << "X is even";
    else
        std::cout << "X is odd";
}

int main()
{
    printEven<11>();
    printEven<42>();
}
```

Kompilierzeitparameter

Check / Verzweigung zur Laufzeit
Könnte aber zur Kompilierzeit bereits ausgewertet werden

Kompilierzeitweig



Laufzeitweig vs. Kompilierzeitweig – Beispiel

- Zur Laufzeit wird ein Zweig mit Sprüngen im Maschinencode übersetzt:

C++

```
void printEven(int x)
{
    if ((x % 2) == 0)
        std::cout << "X is even";
    else
        std::cout << "X is odd";
}

int main()
{
    printEven(11);
    printEven(42);
}
```

ASM (vereinfacht)

```
printEven:
    mov eax, [x]          ; Eingabeparameter x laden
    mov edx, 2            ; Divisor auf 2 für die Modulo-Operation setzen
    div edx               ; eax durch edx teilen, Ergebnis in eax, Rest in edx

    test edx, edx         ; Überprüfen, ob Rest (edx) gleich null ist
    jz even              ; Springe zu "even", wenn das Ergebnis null ist (gerade)

    ; Zweig für ungerade Zahlen
    call cout, "X is odd" ; Ausgabe ungerade
    jmp end              ; Springe zum Funktions-Ende

even:
    ; Zweig für gerade Zahlen
    call cout, "X is even" ; Ausgabe gerade
```

→ Modulo-Berechnung und Sprünge zur Laufzeit

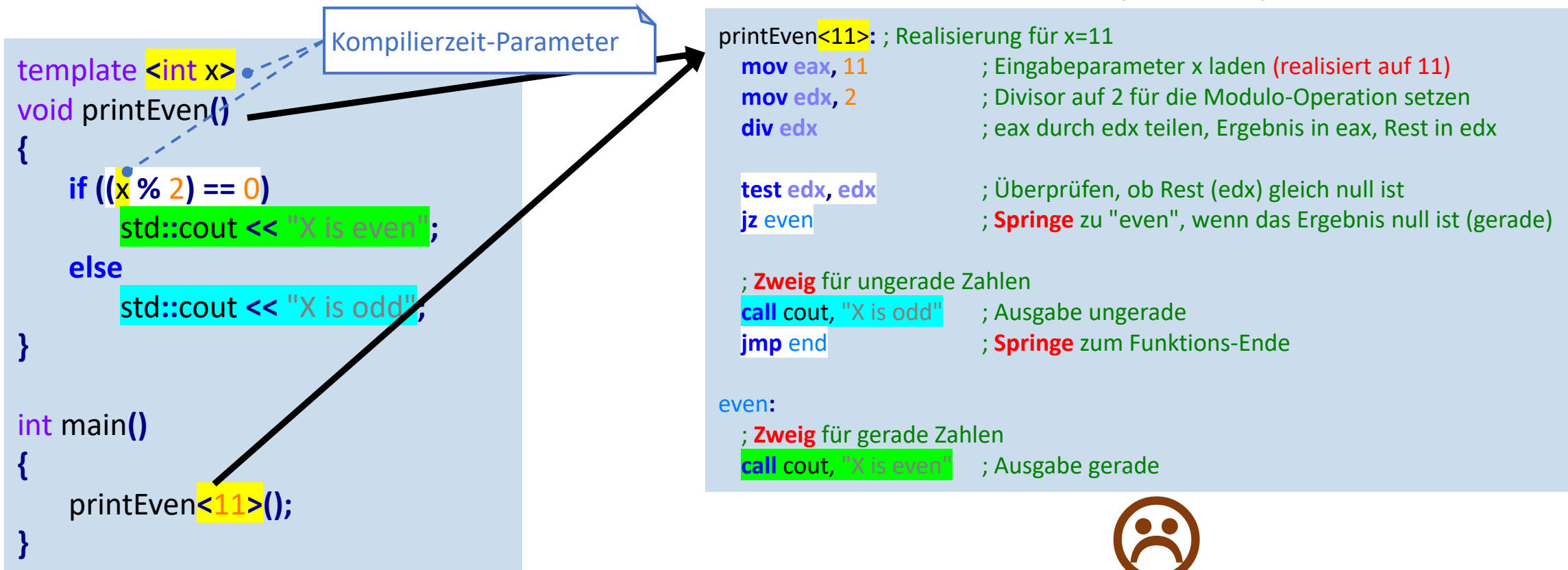
Kompilierzeitweige



Laufzeitweig vs. Kompilierzeitweig – Beispiel

- Erhält eine Funktion einen Parameter, der bereits zu Kompilierzeit bekannt ist, wird dieser trotzdem zur Laufzeit überprüft

ASM (vereinfacht)



Kompilierzeitweige



Laufzeitweig vs. Kompilierzeitweig – Beispiel

- Mit einem **C++17** Kompilierzeitweig lässt sich die Abfrage zur Kompilierzeit erzwingen

ASM (vereinfacht)

```
template <int x>
void printEven()
{
    if constexpr ((x % 2) == 0)
        std::cout << "X is even";
    else
        std::cout << "X is odd";
}

int main()
{
    printEven<11>();
    printEven<42>();
}
```

Kompilierzeitweig
durch constexpr

```
printEven<11>: ; Realisierung für x=11
; Direkte Ausgabe, keine Berechnungen
; da 11 bereits durch Compiler als ungerade ausgewertet wurde!
call cout, "X is odd" ; Ausgabe ungerade
; keine Sprünge!

printEven<42>: ; Realisierung für x=42
; Direkte Ausgabe, keine Berechnungen
; da 42 bereits durch Compiler als gerade ausgewertet wurde!
call cout, "X is even" ; Ausgabe gerade
; keine Sprünge!
```



→ Kompilierzeitweige erlauben deutliche Optimierungen von Maschinencode, bei Nutzung von Kompilierzeitkonstanten

Kompilierzeitweige



Laufzeitweig vs. Kompilierzeitweig – Beispiel

- Zusatz: Wäre die ganze Funktion hier **constexpr** deklariert worden, könnte der Compiler die Befehle direkt übersetzen

ASM (vereinfacht)

```
template <int x>
constexpr void printEven()
{
    if constexpr ((x % 2) == 0)
        std::cout << "X is even";
    else
        std::cout << "X is odd";
}

int main()
{
    printEven<11>();
    printEven<42>();
}
```

Kompilierzeitfunktion

```
call cout, "X is odd" ; Ausgabe ungerade
call cout, "X is even" ; Ausgabe gerade
```

Direkte
Übersetzung ohne
ASM-Funktionen

Kompilierzeitweige



Kompilierung

- Kompilierzeitweige werden nur dann kompiliert, wenn sie durch eine konkrete **Realisierung** betreten werden
 - Das erlaubt das Schreiben von Code, der nur im Kontext des Zweiges funktionieren kann
- Kompilierzeitweige sind besonders hilfreich bei Typ-abhängigen Abfragen
 - Diese lassen sich in einer Funktion kombinieren, ohne Template-Spezialisierungen (z.B. mittels SFINAE (= Substitution Failure Is Not An Error; z.B. mit `std::enable_if`) erstellen zu müssen

Kompilierzeitweige

Kompilierung – Beispiel



- Beispiel: Eine Funktion soll abhängig von dem Typen der übergebenen Variable Operationen ausführen

```
template <typename T>
void process(T x)
{
    if constexpr (std::is_convertible_v<T, std::string>)
    {
        std::cout << "5tes Zeichen ist: " << x[4];
    }
    else if constexpr (std::is_integral_v<T>)
    {
        std::cout << "Quadrat ist: " << (x * x);
    }
    else
        std::cout << "Typ kann nicht verarbeitet werden";
}
```

```
int main()
{
    process("Hello World");
    process(42);
    process(std::vector<int>{1,2,3});
}
```

```
5tes Zeichen ist: o
Quadrat ist: 1764
Typ kann nicht verarbeitet werden
```

Kompilierzeitweige

Kompilierung – Beispiel



```
template <typename T>
void process(T x)
{
    if constexpr (std::is_convertible_v<T, std::string>)
    {
        std::cout << "5tes Zeichen ist: " << x[4];
    }
    else if constexpr (std::is_integral_v<T>)
    {
        std::cout << "Quadrat ist: " << (x * x);
    }
    else
    {
        std::cout << "Typ kann nicht verarbeitet werden";
    }
}
```

Kompilierzeitweig für
Zeichenketten (strings)

Kompilierzeitweig für
Ganzzahlen

Kompilierzeitweig für
alle anderen Typen

Es werden nur die Anweisungen in den jeweiligen Kompilierzeitweigen kompiliert und ausgeführt!
Sonst würde die Funktion eine Menge Compilerfehler enthalten (z.B. Array-Zugriff auf Ganzzahl)

Kompilierzeitweige

Kompilierung – Beispiel



- Ohne Kompilierzeitweige müssen für gleiches Verhalten drei unterschiedliche Template-Spezialisierungen definiert werden:

1. SFINAE für Zeichenketten (strings)

```
template<typename T>
std::enable_if_t<std::is_convertible_v<T, std::string>>
process(T x)
{
    std::cout << "5tes Zeichen ist: " << x[4];
}
```



2. SFINAE für Ganzzahlen

```
template<typename T>
std::enable_if_t<std::is_integral_v<T>>
process(T x)
{
    std::cout << "Quadrat ist: " << (x * x);
}
```

3. SFINAE für alle anderen Typen

```
template<typename T>
std::enable_if_t<!std::is_convertible_v<T, std::string> && !std::is_integral_v<T>>
process(T x)
{
    std::cout << "Typ kann nicht verarbeitet werden";
}
```

Kompilierzeitweige

Zusammenfassung



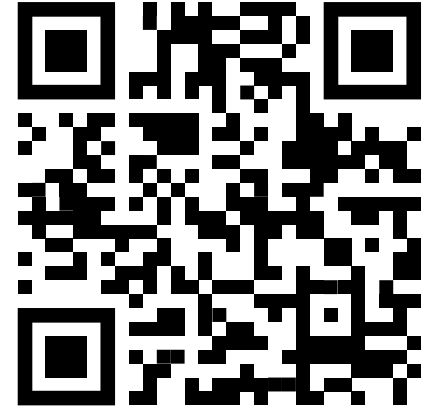
- Mit **Kompilierzeitweigen** können Funktionen optimiert werden, die Kompilierzeitparameter nutzen, da die Auswertung nicht zur Laufzeit erfolgt
- Es werden nur die Zweige kompiliert und ausgewertet, die durch die jeweiligen Realisierungen einer Funktion betreten werden
 - Dadurch wird die **Typ-abhängige Verarbeitung** von Parametern deutlich vereinfacht

Kompilierzeitweige

Verständnisfragen

PIN: 774170

<https://poll.hs-kempten.de/poll/>



■ Was sind Vorteile von Kompilierzeitweigen?

- a) Sie reduzieren den Rechenaufwand für Zweigauswertungen zur Laufzeit.
- b) Sie verhindern die Kompilierung von Code in nicht-gewählten Zweigen.
- c) Sie stellen die Korrektheit des Codes in allen verfügbaren Zweigen sicher.
- d) Sie können Template-Spezialisierungen für Typ-Abfragen ersetzen.

■ WAHR oder FALSCH?

- Kompilierzeitweige können nur auf Kompilierzeitparameter angewendet werden.
- Der else-Zweig eines Kompilierzeitweigs muss constexpr deklariert werden, wenn der if-Zweig constexpr deklariert wurde.
- Der Code eines Kompilierzeitweigs wird nur kompiliert, wenn er durch mindestens eine Funktionsrealisierung betreten wird.
- Der Compiler setzt automatisch Kompilierzeitweige ein, wenn eine if-Bedingung zur Kompilierzeit ausgewertet werden kann.